Programación Orientada a Objetos

Profesor: Dr. Samuel Sepúlveda Cuevas
Ayudante: Jesús Tapia-Martin

02 de octubre 2026

Las siguientes actividades buscan reconocer y justificar las dependencias y asociaciones entre clases. Primero se propondrán las relaciones a partir de un enunciado y luego se identificarán en código Java. Además, se revisará la aplicación del principio de responsabilidad única (SRP).

> **Formato de entrega:** Entregue un único PDF con solo las respuestas en texto y sus justificaciones, indicando la actividad y el número de pregunta. No incluya imágenes ni capturas de pantalla.

1. Actividades

1.1 Actividad 1: relaciones a partir del enunciado

En esta actividad deberá indicar las relaciones que faltan en el diagrama de clases y justificar sus decisiones a partir del siguiente caso.

Una biblioteca registra los préstamos de libros a sus socios. En particular, su funcionamiento es el siguiente:

- Cada préstamo **guarda un socio y un libro**.
- Al devolver el libro, **el socio y el libro siguen registrados**. El préstamo se conserva en el historial.
- Al devolver el libro, el préstamo se marca como devuelto. La multa es de $500 por cada día de atraso. Si no hay atraso, no hay multa.
- El comprobante se imprime con la impresora disponible en ese momento. **El préstamo no guarda la impresora**.

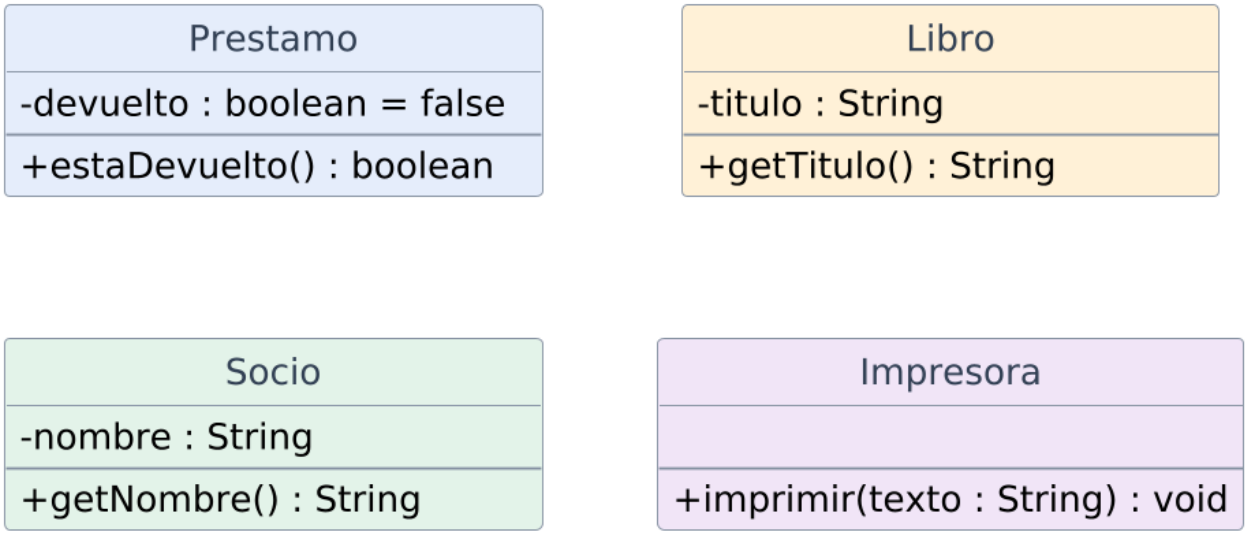


Figura 1: Diagrama de clases sin relaciones.

1. ¿Qué relaciones deberían incorporarse al diagrama? Indique las clases involucradas, el tipo de relación y justifique su elección.
2. Para cada asociación propuesta, indique sus multiplicidades y explique qué significan en el caso.
3. ¿Bastaría representar todas las relaciones como dependencias para describir el caso? Justifique con el enunciado.
4. ¿Qué objetos debería guardar `Prestamo` como atributos? Justifique.
5. ¿Qué parámetros y tipos propondría para `devolver`? Explique para qué sirve cada uno.

1.2 Actividad 2: relaciones en el código

En esta actividad identificará dónde se reflejan las asociaciones y dependencias en el código del caso anterior. También revisará si la clase `Prestamo` aplica SRP.

Código 1: Clase Prestamo

```
class Prestamo {
    private Socio socio;
    private Libro libro;
    private boolean devuelto;

    public Prestamo(Socio socio, Libro libro) {
        this.socio = socio;
        this.libro = libro;
```

```
        this.devuelto = false;
    }

    public void devolver(int diasAtraso, Impresora impresora) {
        int multa = diasAtraso > 0 ? diasAtraso * 500 : 0;
        devuelto = true;

        String comprobante =
            "Socio: " + socio.getNombre()
            + "\nLibro: " + libro.getTitulo()
            + "\nMulta: $" + multa;

        impresora.imprimir(comprobante);
    }

    public boolean estaDevuelto() {
        return devuelto;
    }
}
```

Código 2: Clases Socio, Libro e Impresora

```
class Socio {
    private String nombre;

    public Socio(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }
}

class Libro {
    private String titulo;

    public Libro(String titulo) {
        this.titulo = titulo;
    }

    public String getTitulo() {
        return titulo;
    }
}

class Impresora {
    public void imprimir(String texto) {
        System.out.println(texto);
    }
}
```

Responda las siguientes preguntas a partir del código:

1. ¿Dónde se reflejan las asociaciones y dependencias en el código? Señale los atributos o métodos, las clases involucradas y justifique cada relación.
2. ¿Cuántas asociaciones y dependencias hay entre las cuatro clases?
3. ¿Existe una relación directa entre `Socio` y `Libro`? Justifique con el código.
4. ¿La clase `Prestamo` aplica SRP? Justifique según sus razones para cambiar y proponga una separación si la considera necesaria.

> **Nota:** Considere solo las cuatro clases del caso y cuente cada relación una sola vez.